

Informe de Arquitectura — Sistema de Agendamiento de Turnos

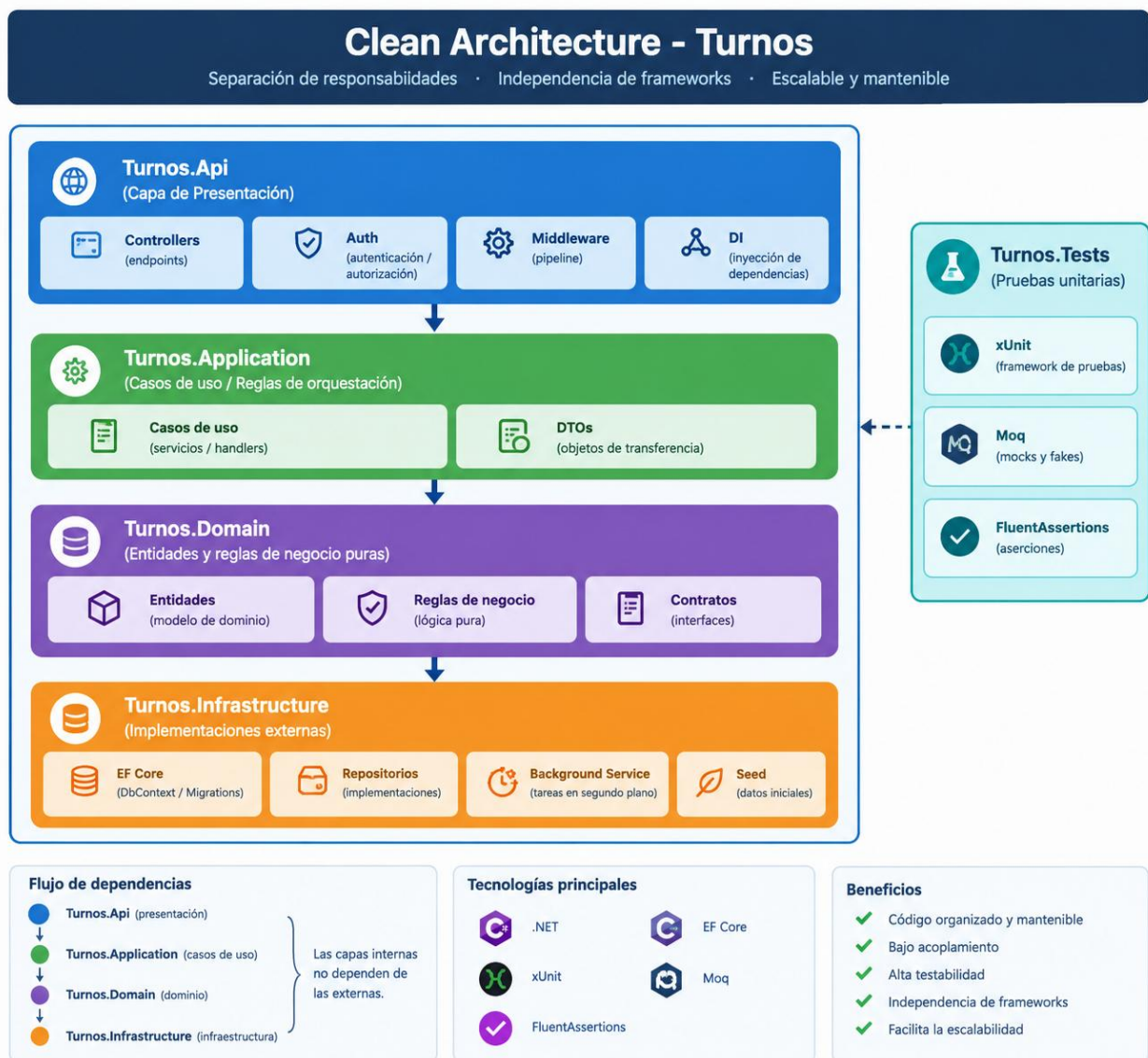
1. Objetivo

Permitir que los clientes de la entidad bancaria agenden turnos de atención en cualquier sucursal desde una app móvil o la web, usando su número de cédula, con un límite de 15 minutos para activar el turno al llegar físicamente, un máximo de 5 turnos solicitados por día por cédula, y expiración automática de los turnos no activados.

2. Visión general de la arquitectura

La solución se divide en dos proyectos independientes que se comunican por HTTP/REST:

2.1 Backend — Clean Architecture / capas por responsabilidad



La dependencia siempre apunta hacia el **Dominio**: Api e Infrastructure dependen de

Application, que a su vez depende de Domain; Domain no depende de nada. Esto permite:

- Sustituir el motor de base de datos (SQL Server, PostgreSQL, etc.) sin tocar reglas de negocio.
- Probar la lógica de agendamiento sin base de datos real (mocks de `ITurnoRepository` / `ISucursalRepository`).

2.2 Patrones de diseño aplicados

Patrón	Dónde	Por qué
Repository	<code>ITurnoRepository</code> , <code>ISucursalRepository</code> + implementaciones EF Core	Aísla el acceso a datos del resto de la app; facilita pruebas con dobles.
Service Layer / Application Service	<code>TurnoService</code> , <code>SucursalService</code>	Concentra los casos de uso (crear, activar, listar, actualizar) y coordina repositorios + reglas de dominio.
Rich Domain Model	Entidad <code>Turno</code>	Las transiciones de estado (Activar, Cancelar, MarcarAtendido, IntentarExpirar) viven en la entidad, no en el service, evitando un modelo con solo datos y sin lógica, garantizando invariantes (p. ej. no se puede activar un turno ya atendido).
Dependency Injection	<code>Program.cs</code> (contenedor nativo de .NET)	Bajo acoplamiento entre capas; permite reemplazar implementaciones en pruebas.
Middleware / Pipeline	<code>ExceptionMiddleware</code>	Traduce excepciones de dominio a respuestas HTTP consistentes sin repetir try/catch en cada controller.
Background Worker (Hosted Service)	<code>ExpiracionTurnosService</code>	Expira turnos vencidos de forma proactiva (cada 30s), garantizando la regla de los 15 minutos incluso si nadie vuelve a consultar el turno.
DTO / Anti-Corruption Layer	<code>Turnos.Application.DTOS</code>	La API nunca expone las entidades de dominio directamente; controla qué información sale y desacopla el contrato REST de la persistencia.

2.3 Frontend — Angular por funcionalidades

El frontend utiliza una arquitectura **feature-based** con **Standalone Components**, sin `NgModules`. Las funcionalidades se agrupan por pantalla y las responsabilidades compartidas se concentran en `core`:

```
src/app/
├── core/
│   ├── guards/           (protección de rutas)
│   ├── interceptors/     (JWT en peticiones HTTP)
│   ├── models/           (modelos TypeScript)
│   └── services/         (AuthService, TurnoService, SucursalService)
└── features/
    ├── login/
    ├── agendar-turno/
    ├── lista-turnos/
    └── turno-detalle/
```

Cada funcionalidad representa una pantalla(componente) y mantiene su lógica TypeScript y su plantilla HTML separadas. Las rutas utilizan `loadComponent` para cargar los componentes bajo demanda, reduciendo el peso inicial de la aplicación al abrirse.

Convenciones de nombres del frontend

- Archivos y carpetas en **kebab-case**: `agendar-turno` y `agendar-turno.component.ts`.
- Clases en **PascalCase**: `AgendarTurnoComponent` y `TurnoService`.
- Selectores de componentes con el prefijo `app-`: `app-agendar-turno`.
- Servicios con el sufijo `.service.ts`, guards con `.guard.ts` e interceptores con `.interceptor.ts`.
- Las pruebas se nombran con el sufijo `.spec.ts` y se ubican junto al código que validan.

3. Modelo de datos

Motor de base de datos

La solución utiliza **SQL Server** (desplegado como **Azure SQL Database**) a través de Entity Framework Core (`Microsoft.EntityFrameworkCore.SqlServer`), configurado con `UseSqlServer(...)` en `Program.cs`. SQL Server ofrece soporte real de escrituras concurrentes, bloqueos a nivel de fila y transacciones, lo que resulta clave para generar los códigos de turno de forma segura bajo concurrencia (ver sección 6).

El acceso a datos está aislado en `Turnos.Infrastructure`, por lo que la aplicación no queda acoplada a un motor concreto: migrar a otro proveedor (p. ej. PostgreSQL con `UseNpgsql(...)`) implicaría cambiar solo la configuración del `DbContext` y la cadena de conexión; las entidades, repositorios, servicios y reglas de negocio permanecen iguales.

La base de datos se llama **dbturnos** y contiene tres tablas, todas mapeadas por Entity Framework Core a partir de las entidades de `Turnos.Domain` (ver

TurnosDbContext.OnModelCreating).

3.1 Tabla Sucursales

Guarda las sucursales físicas donde un cliente puede ser atendido.

Columna	Tipo	Restricciones	Descripción
Id	int	PK, autoincremental	Identificador de la sucursal.
Nombre	varchar(150)	requerido	Nombre visible de la sucursal.
Direccion	varchar(250)	requerido	Dirección física.
Ciudad	varchar(100)	requerido	Ciudad donde está ubicada.
Activa	bit	requerido, default true	Solo las sucursales activas permiten crear turnos nuevos.

3.2 Tabla Turnos

Es la tabla central del sistema: cada fila es un turno solicitado por un cliente para una sucursal específica.

Columna	Tipo	Restricciones	Descripción
Id	uniqueidentifier (GUID)	PK	Identificador único del turno, generado en el servidor al crearlo.
CodigoTurno	varchar(30)	único, requerido	Código consecutivo legible que se le muestra al cliente y al empleado (ej. sucursal + número).
Cedula	varchar(20)	requerido	Cédula del cliente dueño del turno.
SucursalId	int	FK a Sucursales.Id	Sucursal donde se atenderá el turno.
FechaHoraCreacion	datetime	requerido	Momento (hora Colombia) en que se solicitó el turno.
FechaHoraExpiracion	datetime	requerido	FechaHoraCreacion + 15 minutos; límite para activarlo.
FechaHoraActivacion	datetime	opcional (nullable)	Se llena solo cuando el cliente activa el turno al llegar a la sucursal.
Estado	varchar(20)	requerido, guardado	Uno de: Pendiente,

Columna	Tipo	Restricciones	Descripción
		como texto	Activado, Atendido, Expirado, Cancelado.

Se guarda el estado como texto (Pendiente, Activado, etc.) en lugar de un número, para que la tabla se pueda leer e inspeccionar directamente en la base de datos sin necesidad de recordar a qué número corresponde cada estado.

Relación: `Turnos.SucursalId` → `Sucursales.Id` (muchos turnos pertenecen a una sucursal). El borrado está restringido (`DeleteBehavior.Restrict`): no se puede eliminar una sucursal que ya tenga turnos asociados, para no perder el historial.

Índices:

- `UX_Turnos_CodigoTurno` (único) sobre `CodigoTurno`: evita a nivel de base de datos que existan dos turnos con el mismo código, incluso si algo fallara en la generación del consecutivo.
- `(Cedula, FechaHoraCreacion)`: acelera la consulta que cuenta cuántos turnos ha pedido una cédula en el día (regla de máximo 5 turnos diarios).
- `(Estado, FechaHoraExpiracion)`: acelera el barrido periódico que busca turnos Pendiente ya vencidos para expirarlos.

3.3 Tabla TurnosConsecutivos

Tabla de apoyo, con una sola fila por sucursal, que lleva el último número de turno asignado en esa sucursal.

Columna	Tipo	Restricciones	Descripción
<code>SucursalId</code>	int	PK	Sucursal a la que pertenece el contador.
<code>UltimoConsecutivo</code>	int	requerido	Último número de turno entregado en esa sucursal.

Esta tabla existe para poder incrementar el consecutivo de forma atómica (`MERGE ... WITH (HOLDLOCK)`, ver sección 6) sin tener que calcularlo leyendo y contando los turnos existentes, lo cual sería lento y además propenso a duplicados si dos clientes piden turno al mismo tiempo.

4. Reglas de negocio y dónde se implementan

Regla	Implementación
Límite de 15 minutos para activar	<code>Turno.Crear</code> fija <code>FechaHoraExpiracion = ahora +</code>

Regla	Implementación
	15min; <code>Turno.Activar</code> valida el tiempo y lanza <code>TurnoExpiradoException</code> si ya venció.
Expiración automática	<code>ExpiracionTurnosService (BackgroundService) + Turno.IntentarExpirar</code> , corre cada 30s.
Máximo 5 turnos/día por cédula	<code>TurnoService.CrearTurnoAsync</code> consulta <code>CountByCedulaBetweenAsync</code> (turnos no cancelados del día colombiano) antes de crear; si es ≥ 5 lanza <code>LimiteTurnosDiariosException</code> (HTTP 409). El contador se reinicia naturalmente al cambiar de día porque el filtro usa el rango [hoy 00:00, mañana 00:00) de Colombia.
Fecha y hora de negocio	<code>ColombiaClock</code> genera la hora local de Colombia para creación, expiración y activación. El backend compara todas las fechas de turnos con el mismo reloj para evitar diferencias de zona horaria.
Código consecutivo del turno	<code>TurnoService</code> obtiene el siguiente consecutivo por sucursal mediante <code>GetNextConsecutivoAsync</code> , que incrementa de forma atómica un contador dedicado (<code>TurnosConsecutivos</code>) con <code>MERGE ... WITH (HOLDLOCK)</code> , garantizando códigos únicos incluso ante solicitudes concurrentes.
Reintentar tras expirar	El cliente simplemente vuelve a llamar <code>POST /api/turnos</code> ; como el turno expirado no cuenta distinto de uno vigente, solo se bloquea si ya llegó a 5 turnos "vivos" (no cancelados) ese día — incluyendo expirados, que sí cuentan como intento, tal como lo especifica el enunciado ("más de 5 turnos solicitados en el día").
Solo sucursales activas	Se valida <code>Sucursal.Activa</code> en <code>CrearTurnoAsync</code> .

5. Seguridad

- **Autenticación JWT** (`Microsoft.AspNetCore.Authentication.JwtBearer`), con dos flujos:
 - `POST /api/auth/token-cliente { cedula }` → token con rol `Cliente` y el claim `cedula`. Representa que el cliente ya pasó la autenticación en

su app y solo necesita una sesión para consumir la API de turnos.

- **Autorización por rol y por dueño del recurso:** todos los endpoints de `TurnosController` exigen `[Authorize]`; un cliente solo puede crear/ver/ activar **sus propios** turnos (se compara el `claim cedula` del token con la cédula del recurso); `PUT /api/turnos/{id}` (marcar `Atendido/Cancelado`) está restringido a `[Authorize(Roles = "Empleado")]`.
- **CORS** restringido a los orígenes configurados (`http://localhost:4200` en desarrollo).
- La llave de firma JWT, credenciales y cadenas de conexión están en `appsettings.json` solo como ejemplo de prueba técnica; en un entorno real irían en un vault / variables de entorno / `dotnet user-secrets`.

6. Eficiencia y escalabilidad

- **API sin estado + JWT:** el servidor no guarda sesión en memoria; cada petición llega con su token y puede ser atendida por cualquier instancia del backend. Esto permite subir varias copias de la API detrás de un balanceador y enviar tráfico entre ellas sin que el usuario tenga que ir siempre a la misma máquina.
- **Concurrencia en la base de datos:** se usa **SQL Server** (Azure SQL), con soporte real de escrituras concurrentes. El consecutivo del código de turno se genera mediante una operación atómica (`MERGE ... WITH (HOLDLOCK)`) dentro de una transacción, que serializa el incremento por sucursal y evita códigos de turno duplicados bajo carga concurrente; el índice único `UX_Turnos_CodigoTurno` actúa como garantía final (ver `CONCURRENCIA_Y_ESCALABILIDAD.md`).
- **Índices** en las columnas más consultadas (ver sección 3) para mantener las validaciones de límite diario y el barrido de expiración incluso con el crecimiento de la tabla `Turnos`.
- El **worker de expiración** corre en un scope propio y en lote (batch), evitando bloquear el hilo de peticiones HTTP.
- Cada creación de turno es independiente (cada llamada produce un turno nuevo con su propio identificador único GUID), lo que facilita reintentos seguros desde el cliente ante fallos de red.

7. Front-end (Angular)

- **Standalone components** (Angular 17+), sin `NgModules`, con *lazy loading* por ruta (`loadComponent`) para minimizar el peso inicial de carga de la aplicación.
- **Capa core:** `AuthService` (sesión con signals), `TurnoService`, `SucursalService` (HTTP), `authInterceptor` (adjunta el JWT a cada petición) y `authGuard` (protege rutas autenticadas).
- **Capa features:** un componente por pantalla (`login`, `agendar-turno`, `lista-turnos`, `turno-detalle`), cada uno con su HTML separado, siguiendo separación de

responsabilidades.

- El componente de agendamiento muestra un **cronómetro en vivo** de los 15 minutos y deshabilita "Activar turno" cuando llega a cero, reflejando en UI la misma regla de negocio que protege el backend.

8. Pruebas

- **Backend** (`Turnos.Tests`, xUnit + Moq + FluentAssertions): reglas de creación (límite diario, sucursal inactiva/inexistente), activación dentro/fuera de tiempo, transición de estados inválida, y reglas puras de la entidad `Turno`.
- **Frontend** (Jasmine/Karma, generado por Angular CLI): `AuthService`, `TurnoService` (peticiones HTTP simuladas con `HttpTestingController`) y el componente `AgendarTurnoComponent`.

9. Supuestos y alcance

- **No se implementó el registro ni la gestión de usuarios administradores**, debido a que, de acuerdo con el enunciado de la prueba técnica, esta funcionalidad se encuentra fuera del alcance definido para la solución.
- Se usa **SQL Server** (Azure SQL Database) como motor de persistencia; el acceso a datos aislado en `Turnos.Infrastructure` permite migrar a otro proveedor cambiando solo la configuración del `DbContext` (ver sección 6).

10. Despliegue en Azure y arquitectura de la solución

La prueba técnica fue desplegada en Azure para validar el flujo real de una aplicación cliente-servidor con frontend estático, API en un plan de App Service y base de datos SQL Server gestionada.

10.1 URLs desplegadas

- **Azure Static Web App (frontend)**: <https://gray-field-03d5bbe0f3.azurestaticapps.net/login>
- **Azure App Service (API - Swagger)**: <https://turnos-backend-hpgygtvvgpcdbbf.westus3-01.azurewebsites.net/swagger/index.html>

10.2 Arquitectura desplegada

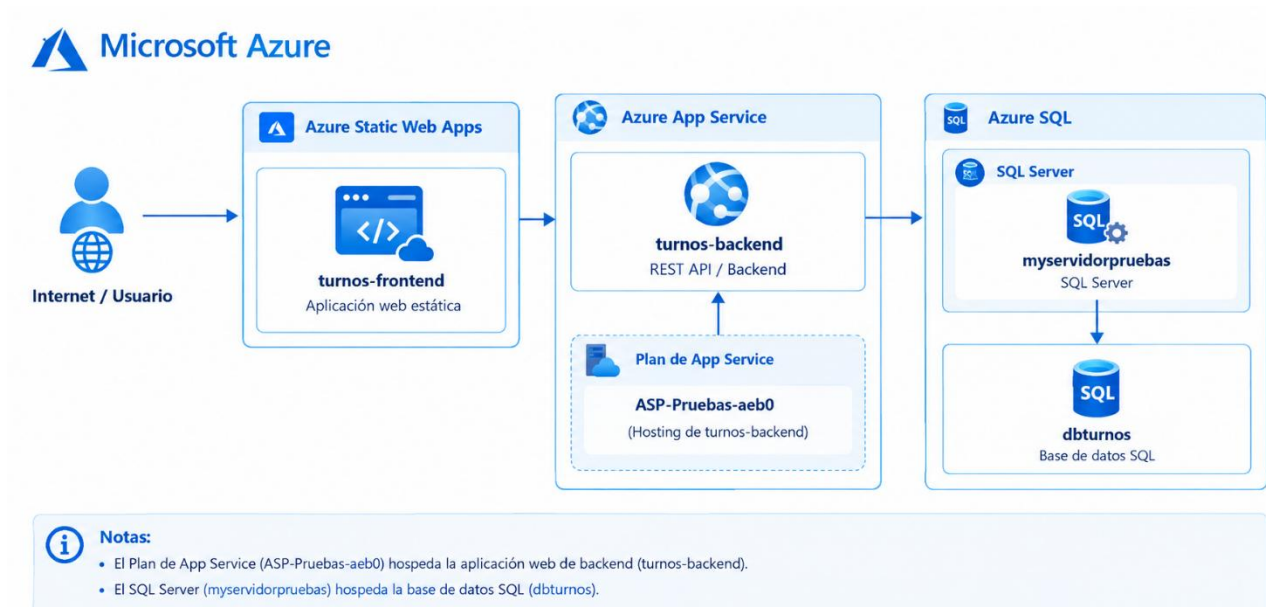
La solución se compone de estos componentes:

- **Azure Static Web Apps:** aloja el frontend Angular (turnos-frontend) y sirve la aplicación web de forma estática.
- **Azure App Service:** hospeda la API REST (turnos-backend), que expone los endpoints de autenticación, gestión de turnos y validaciones de negocio.
- **Azure SQL Server / Azure SQL Database:** almacenas sucursales, turnos, consecutivos y toda la información transaccional requerida por la aplicación.

La comunicación sigue este flujo:

1. El usuario accede a la aplicación web desde Internet.
2. El frontend consume la API a través de HTTPS con JWT.
3. La API valida el token, aplica reglas de negocio y consulta la base de datos.
4. Azure SQL guarda y devuelve la información persistente de turnos y sucursales.

10.3 Imagen de arquitectura en Azure



- **Internet / Usuario:** es el punto de entrada desde donde el cliente accede a la aplicación.
- **Azure Static Web Apps:** contiene la interfaz web del sistema; es el sitio público que entrega la experiencia frontend sin necesidad de un servidor de aplicación para renderizar páginas dinámicas.
- **Azure App Service:** alberga el backend .NET, con la API REST y la lógica de negocio. Se conecta con la base de datos y aplica la autenticación y la autorización del sistema.
- **Azure SQL Server:** gestiona la persistencia de datos. El contenedor de la base de datos dbturnos guarda la información de los turnos y las sucursales.
- **Plan de App Service:** indica que la API está desplegada en un entorno de hosting administrado por Azure, pudiendo escalar y mantener la aplicación sin gestionar infraestructura a nivel de sistema operativo.

10.4 Beneficios de este despliegue

- **Escalabilidad:** el frontend y la API pueden escalar independientemente según la carga.
- **Mantenimiento simplificado:** Azure se encarga del hosting y de la infraestructura base.
- **Seguridad:** la API se expone como servicio y la autenticación se maneja con JWT, con CORS y reglas de autorización para proteger los recursos.
- **Disponibilidad:** la separación de responsabilidades entre frontend, API y base de datos facilita despliegues y mantenimiento más seguros y predecibles.

11. Concurrencia y escalabilidad — Generación de turnos

11.1 Requisito de la prueba

Eficiencia y escalabilidad: considerar la eficiencia y escalabilidad de la solución, especialmente en términos de manejo de solicitudes concurrentes y escalabilidad horizontal.

El defecto rompía este requisito: la solución no era segura ante concurrencia ni apta para escalar horizontalmente (varias instancias del API agravarían el problema porque cada una mantenía su propio ciclo lectura-escritura).

11.2 Solución adoptada — Incremento atómico en base de datos

Se reemplazó el read-modify-write por una **única operación atómica con bloqueo** en la base de datos, ejecutada dentro de una transacción:

```
MERGE dbo.TurnosConsecutivos WITH (HOLDLOCK) AS target
USING (SELECT @sucursalId AS SucursalId) AS src
ON target.SucursalId = src.SucursalId
WHEN MATCHED THEN
    UPDATE SET UltimoConsecutivo = target.UltimoConsecutivo + 1
WHEN NOT MATCHED THEN
    INSERT (SucursalId, UltimoConsecutivo) VALUES (src.SucursalId, 1);
```

Claves de la solución:

- **MERGE (upsert):** en una sola sentencia crea la fila si no existe o la incrementa si ya existe, cubriendo el primer turno de cada sucursal.
- **WITH (HOLDLOCK):** toma un bloqueo de rango sobre la fila de la sucursal y lo mantiene hasta el COMMIT, serializando el incremento. Dos peticiones concurrentes ya no pueden obtener el mismo valor.
- **El índice único UX_Turnos_CodigoTurno se conserva** como garantía final: es la última línea de defensa ante cualquier fallo de la aplicación.

Por qué esta opción

Criterio	Beneficio
Eficiencia	Una sola ida a la base de datos; sin transacciones largas ni contención global.

Criterio	Beneficio
Escalabilidad horizontal	El consecutivo vive en la base de datos, no en memoria del proceso: N instancias del API tras un balanceador comparten el mismo contador de forma consistente.
Simplicidad	No requiere cambiar el modelo de datos ni introducir componentes externos.

11.3 Consideraciones de escalabilidad horizontal

- **API stateless:** el estado (consecutivo) reside en la base de datos, por lo que se pueden ejecutar múltiples instancias del API sin coordinación adicional.
- **I/O asíncrono:** todos los accesos a datos usan `async/await`, liberando hilos durante la espera de la base de datos y mejorando el throughput.
- **Servicio de expiración en segundo plano:** opera de forma idempotente para no interferir cuando corran varias instancias.
- **Pool de conexiones:** se mantiene el pooling de EF Core / SQL Server para soportar concurrencia sin agotar conexiones.

12. Pruebas unitarias

Se implementaron pruebas unitarias tanto en el backend como en el frontend, cubriendo las reglas de negocio y los flujos principales de la aplicación (creación de turnos, activación, límites diarios y consumo de la API).

12.1 Backend

- **Framework:** `xUnit`, con `Moq` para simular los repositorios (`ITurnoRepository`, `ISucursalRepository`) y `FluentAssertions` para aserciones más legibles.
- **Ubicación:** proyecto `Turnos.Tests`.
- `TurnoEntityTests.cs`: valida las reglas puras de la entidad `Turno`, por ejemplo que un turno pendiente vencido se marque `Expirado` (`IntentarExpirar_SiVencioYSigueEnPendiente_DeberiaMarcarExpirado`) y que no se pueda cancelar un turno ya atendido (`Cancelar_CuandoYaFueAtendido_DeberiaLanzarExcepcion`).
- `TurnoServiceTests.cs`: valida los casos de uso orquestados por `TurnoService`, entre ellos:
 - Creación de un turno válido con estado `Pendiente` y código consecutivo.
 - Generación del siguiente código de turno a partir del consecutivo de la sucursal.
 - Rechazo al intentar crear un sexto turno el mismo día para la misma cédula (límite

diario).

- Rechazo al crear un turno para una sucursal inexistente o inactiva.
- Activación de un turno dentro del tiempo límite.
- Cambios de estado válidos e inválidos (por ejemplo, no se puede marcar Atendido un turno que sigue Pendiente).

12.2 Frontend

- **Framework:** Jasmine como framework de pruebas y Karma como test runner (generado por Angular CLI), ejecutando en ChromeHeadless.
- `auth.service.spec.ts`: valida que el servicio autentique al cliente y guarde la sesión, y que `logout` limpie la sesión correctamente.
- `turno.service.spec.ts`: valida las peticiones HTTP del servicio de turnos usando `HttpTestingController`, simulando la creación de un turno (`POST /turnos`), su activación (`POST /turnos/{id}/activar`) y el listado filtrado por cédula.
- `agendar-turno.component.spec.ts`: valida el componente de agendamiento, incluyendo la carga de sucursales activas al iniciar, el mensaje de error cuando se agenda sin seleccionar sucursal, y el formato del cronómetro de 15 minutos (`formatoTiempo`).
- `app.component.spec.ts`: prueba básica de creación de la aplicación.

12.3 Cobertura de las principales funcionalidades

Funcionalidad	Backend	Frontend
Crear turno	✓ (válido, límite diario, sucursal inexistente/inactiva)	✓ (servicio y componente)
Activar turno	✓ (dentro del tiempo límite)	✓ (servicio)
Expirar turno	✓ (entidad Turno)	— (regla protegida solo en backend)
Cambiar estado (Atendido/Cancelado)	✓ (transiciones válidas e inválidas)	—
Autenticación	— (fuera del alcance de las pruebas actuales)	✓ (login y logout)
Listado de turnos	✓ (a través de <code>TurnoServiceTests</code>)	✓ (filtro por cédula)

13. Procesamiento en segundo plano y evolución para producción

Para esta prueba técnica se eligió un BackgroundService de ASP.NET Core, implementado mediante ExpiracionTurnosService, para ejecutar periódicamente la expiración de los turnos pendientes cuyo tiempo de activación ya terminó. El servicio corre dentro del backend, utiliza el contenedor de inyección de dependencias y crea un scope propio para acceder a los repositorios y a la base de datos. Esta alternativa es suficiente para demostrar la regla de negocio, mantener la solución sencilla y evitar dependencias adicionales.

En un entorno productivo, especialmente con varias instancias del backend o con necesidades avanzadas de operación, se debería evaluar una solución de procesamiento programado más robusta. Algunas alternativas son:

- **Hangfire:** permite programar y persistir trabajos, administrar reintentos, consultar historiales y monitorear la ejecución desde un panel web. Requiere configurar un almacenamiento compartido y controlar la ejecución distribuida para evitar que varias instancias procesen el mismo trabajo indebidamente.
- **Azure Functions con Timer Trigger:** ejecuta la tarea según una expresión programada y delega la operación y el escalado a Azure. Es una opción apropiada si se desea desacoplar la expiración del ciclo de vida del API, aunque requiere configurar la función, la identidad de acceso y los recursos de Azure correspondientes.

La elección entre estas alternativas dependería del volumen de turnos, la frecuencia de ejecución requerida, las necesidades de reintento y monitoreo, la estrategia de despliegue y los costos operativos. En cualquiera de los casos, la operación de expiración debe poder ejecutarse repetidamente de forma segura ante ejecuciones concurrentes, de modo que procesar un turno más de una vez no produzca cambios incorrectos.